

UL Configured Grant Type 2 in OCUDU

Implementation proposal: current state, required changes, parameters and block diagrams

Item	Value
Document	CG Type 2 implementation proposal, version 2 (supersedes "UL Configured Grant Type 2: Parameter Inventory and DCI 0_1 Integration Proposal")
Code baseline	OCUDU 6153e7bf2 (2026-08-25, "cmake: fix AOCL-FFTZ library find instructions"). All file:line references point to this commit.
Author	Surabhi P
Date	24 September 2026
Status	Design proposal. No code has been written; every claim about the current code was checked against the source at the baseline commit.
Spec references	TS 38.321 §5.4.1, §5.8.2, §6.1.3.7 · TS 38.213 §10.2 · TS 38.212 §7.3.1.1.2 · TS 38.214 §6.1.2 · TS 38.331 ConfiguredGrantConfig

Colour code used in every table and diagram

NEW: code that does not exist and must be written. **MODIFY:** existing function or struct that must change.

REMOVE: an existing Type-1-only guard or assert that must be deleted or relaxed.

UNCHANGED: existing code that Type 2 reuses as-is.

VERIFY: outside OCUDU (the OAI UE) or needs confirmation before relying on it. **OUTPUT:** result objects and hand-off points.

Contents

1. Executive summary (1.1 Corrections to the first draft)
2. Background: how CG Type 2 works (3GPP)
3. Lab setup and baseline evidence
4. What exists today (as-is): Type 1 path, the 14 places that assume Type 1, reusable parts
5. Parameters involved: ConfiguredGrantConfig, grant fields, DCI 0_1 fields, runtime state, new config
6. Proposed changes: file and function inventory (per figure, with IDs)
7. Block diagrams (Figures 1-4, landscape pages)
8. Design decisions
9. Implementation plan and test plan
10. Risks and open questions
11. Glossary

1. Executive summary

OCUDU already has a working UL Configured Grant (CG) **Type 1** implementation, in which the whole grant comes from RRC. This document proposes how to add **Type 2**, where RRC sends only the semi-static part (periodicity, HARQ count, DMRS, MCS table) and the gNB activates or releases the grant with a DCI format 0_1 whose CRC is scrambled by the CS-RNTI. The proposal was built by reading the code at the baseline commit, not only the earlier draft.

- **No new DCI bit-fields.** `dci_0_1_configuration` and `dci_0_1_pack()` (`dci_packing.h`:780, :953) stay unchanged. Activation and release reuse the existing fields with the reserved values from TS 38.213 §10.2.
- **No new RRC field.** `cg_configuration` already treats an absent `rrc_configured_ul_grant_cfg` as Type 2, and the ASN.1 encoder (`asn1_rrc_config_helpers.cpp`:2951) already omits `rrc-ConfiguredUplinkGrant` when it is absent. CS-RNTI allocation (`mac_controller.cpp`:109) is already type-agnostic.
- **The work is larger than the first draft said.** It is not "one new function plus one branch". The code has **14 places that assume Type 1** (Section 4.2), including three that would crash a Type 2 UE. The full plan touches about 30 source files: **12 new symbols, 27 modified, 3 Type-1 guards removed, and 16 existing pieces reused unchanged** (Section 6.5).
- **Three blockers the first draft missed.** (1) `build_pusch_cs_rnti()` asserts that the RRC grant exists (`sch_pdu_builder.cpp`:680). (2) Five CG-scheduler functions dereference `rrc_configured_ul_grant_cfg.value()`. (3) The CG scheduler has no PDCCH allocator, so it cannot send an activation DCI at all.
- **Recommended approach.** Keep the existing Type 1 resource planning (`cg_type1_res_mng` picks an offset and VRBs) and store the Type 2 grant as a DU-internal template. Send the activation DCI in a slot where $(n + k_2) \bmod P$ equals the planned offset. Route every grant read through one new helper, `get_active_grant()`, so the periodic-occasion code serves both types.
- **Delivery in five phases** (Section 9). Phase 0 is a behaviour-neutral refactor that can merge on its own. The main external risk is whether the OAI UE supports Type 2; check that before end-to-end testing.

1.1 Corrections to the first draft

Each point below was checked against the source at [6153e7bf2](#).

First draft said	What the code shows	Consequence
<code>build_pusch_cs_rnti()</code> : no change expected	Line 680 asserts <code>cg_cfg.rrc_configured_ul_grant_cfg.has_value()</code> with message "RRC-ConfiguredUplinkGrant for CG type 1 not set".	Every Type 2 occasion would hit the assert. The assert must be relaxed (I4).
The scheduler change is one added branch in <code>add_ue_to_wheel()</code>	<code>rem_ue()</code> , <code>reserve_cg_resources()</code> , <code>build_cg_pusch_cfg_params()</code> , <code>allocate_cg_opportunity()</code> and <code>compute_cg_vrbs()</code> all read the RRC grant directly.	Add <code>get_active_grant()</code> and route all five through it (H4, H5, I1, I2).
Add one enum entry <code>cs_rnti_f0_1</code> in <code>dci_info.h</code>	<code>dci_ul_info::type()</code> returns <code>payload.index()</code> , so the enum must match a variant alternative. <code>dci_0_1_configuration</code> is already one alternative, and adding the same type twice breaks <code>std::get<T></code> .	Add a distinct wrapper type (for example <code>dci_0_1_cs_rnti_configuration</code>) and update <code>encode_dci()</code> , the logger and the RNTI-type strings (F4, F5).

First draft said	What the code shows	Consequence
nof_cg_reserved gating is a bug	<code>ue_cell::handle_reconfiguration_request()</code> reserves CG HARQs for any CG. <code>update_harq_reservation()</code> then resets the count to 0 when there is no RRC grant. This is a deliberate Type-1-only guard, consistent with the header remark "Type 2 CG is not currently supported".	Not a defect in today's Type-1-only build, but it blocks Type 2. Drop the guard as part of this work (D3).
Not covered	The config validator rejects Type 2 (<code>scheduler_ue_config_validator.cpp:29</code>). The factory and resource manager always build or assert the RRC grant. There is no CLI option to select the type.	Configuration-path changes A1-A4, B2, B3, B5, C1 and C2.
Not covered	LCID 55 (CG Confirmation MAC CE) has no enum entry. The PDU still parses because the CE is zero-length, but <code>handle_mac_ce()</code> logs "Unhandled LCID" and drops it.	The gNB cannot tell whether activation or release took effect (G2-G5).
Not covered	<code>is_cg_slot()</code> (<code>ue_configuration.cpp:743</code>) says "TODO: support type 2" and only checks config.	Dynamic PUSCH would not be kept out of Type 2 occasions (J1, J2).
Not covered	<code>alloc_ul_pdcch_ue()</code> uses its <code>rnti</code> argument both as the CRC RNTI and for <code>n_RNTI</code> data scrambling (TS 38.211 §7.3.2.3 requires C-RNTI there).	The activation PDCCH must keep C-RNTI for scrambling and use CS-RNTI only for the CRC (F1).

2. Background: how CG Type 2 works (3GPP)

2.1 Type 1 compared with Type 2

Aspect	Dynamic grant	CG Type 1	CG Type 2
Where the grant comes from	A DCI 0_0/0_1 (C-RNTI) for every PUSCH	RRC <code>rrc-ConfiguredUplinkGrant</code>	RRC semi-static part plus one activation DCI 0_1 (CS-RNTI)
Start of occasions	k2 after each DCI	<code>timeDomainOffset</code> relative to SFN 0	The PUSCH slot indicated by the activation DCI ($n + k2$)
Stop	n/a	RRC reconfiguration	Release DCI (CS-RNTI) or RRC reconfiguration
Grant fields (TDRA, FDRA, MCS, antenna ports)	Every DCI	RRC	Activation DCI, then fixed until re-activation
HARQ process ID	Scheduler chooses one	$\text{floor}(\text{CURRENT_symbol} / P) \bmod \text{nrofHARQ-Processes}$	Same formula
UE acknowledgement	n/a	none	CG Confirmation MAC CE (LCID 55) after activation and after release
OCUDU status at baseline	Works (logs in Section 3.1)	Works	Not supported ("Type 2 CG is not currently supported", <code>cg_configuration.h</code>)

2.2 Activation and release validation (TS 38.213 §10.2)

The UE treats a DCI 0_1 as an activation or release only if the CRC is scrambled by CS-RNTI, NDI = 0, and the special fields below have these values. With a single CG configuration, Tables 10.2-3 and 10.2-4 apply:

DCI 0_1 field	Activation (Table 10.2-3)	Release (Table 10.2-4)	Notes
CRC scrambling	CS-RNTI	CS-RNTI	With NDI = 1 and CS-RNTI, the DCI would instead be a retransmission grant for a CG HARQ process. OCUDU does not use that (see Risk R4).
<code>new_data_indicator</code>	0	0	Validation field
<code>harq_process_number</code>	all '0'	all '0'	Validation field when only one CG configuration exists
<code>redundancy_version</code>	'00'	'00'	Validation field
<code>modulation_coding_scheme</code>	CG MCS (becomes the fixed MCS)	all '1' (31)	
<code>frequency_resource</code>	RIV of the CG VRBs (RA type 1)	all '1'	RA type 1 is the only type OCUDU CG supports
<code>time_resource</code> , <code>antenna_ports</code> , <code>precoding</code> , DMRS init	Grant values, stored by the UE	Ignored	Set to template values on release for determinism

2.3 Timing, HARQ and confirmation (TS 38.321)

- **Occasions (§5.8.2).** The N-th occasion is at $[(\text{SFN} \times \text{slotsPerFrame} \times \text{symPerSlot}) + (\text{slot} \times \text{symPerSlot}) + \text{symbol}] = [(\text{SFN_start} \times \text{slotsPerFrame} \times \text{symPerSlot} + \text{slot_start} \times \text{symPerSlot} + \text{symbol_start}) + N \times P] \bmod (1024 \times \text{slotsPerFrame} \times \text{symPerSlot})$. The `_start` values are those of the first PUSCH after (re)initialisation, which is the slot scheduled by the activation DCI. OCUDU's 5120-slot wheel is compatible: every allowed periodicity divides 5120, and 20480 slots per hyper-frame at 30 kHz is a multiple of 5120.
- **HARQ ID (§5.4.1).** HARQ Process ID = $\text{floor}(\text{CURRENT_symbol} / \text{periodicity}) \bmod \text{nrofHARQ-Processes}$, the same for both types (no Rel-16 harq-ProcID-Offset2). The existing `get_harq_id()` (configured_grant_scheduler_impl.cpp:19) already implements it.
- **configuredGrantTimer.** Applies to both types. OCUDU fixes it at $4 \times P$ (`cg_configuration::configured_grant_timer`, used as `cg_harq_timeout`).
- **CG Confirmation MAC CE (§6.1.3.7).** UL-SCH LCID 55 (0b110111), zero payload. The UE triggers it on both activation and release and sends it in the next UL resource for new transmission, usually the first CG occasion.

3. Lab setup and baseline evidence

Parameter	Value
gNB	OCUDU (commit 6153e7bf2), single cell
UE	OAI nr-uesoftmodem, ZMQ device (oai_zmqdevif)
Core network	open5gs, Docker container open5gs_5gc
PCI / Band / ARFCN	1 / n78 / dl_arfcn = 632628 (3489.42 MHz)
Bandwidth / SCS	20 MHz, 30 kHz (51 PRB, numerology 1)
Duplexing / Transport / Mode	TDD / ZMQ RF simulation (no hardware) / Standalone, single UE
Logging	<code>log.mac_level: debug</code> enables both the MAC and SCHED loggers. UL grants appear as <code>format=0_1 type=c-rnti</code> lines.

3.1 Current dynamic-grant behaviour (SCHED debug log)

```
- UL PDCCH: rnti=0x4601 type=c-rnti cs_id=1 ss_id=2 format=0_1 cce=2 al=2 dci: h_id=0 ndi=0 rv=0
mcs=28 tpc=1 dai=3 mimo=0 ant=2
- UL PDCCH: rnti=0x4601 type=c-rnti cs_id=1 ss_id=2 format=0_1 cce=2 al=2 dci: h_id=0 ndi=1 rv=0
mcs=28 tpc=1 dai=3 mimo=0 ant=2
- UL PDCCH: rnti=0x4601 type=c-rnti cs_id=1 ss_id=2 format=0_1 cce=2 al=2 dci: h_id=0 ndi=0 rv=0
mcs=28 tpc=1 dai=3 mimo=0 ant=2
```

Observation	Meaning	Relevance to CG Type 2
<code>h_id</code> fixed at 0, ndi toggles	One UL HARQ process in use, with clean new-transmission toggling	CG HARQ IDs 0..N-1 are reserved for CG (<code>first_non_reserved_harq_id</code>), so dynamic grants move to IDs $\geq N$ once CG is configured
<code>mcs</code> = 28, constant <code>cce/al</code>	Stable link adaptation on the ZMQ channel	Type 2 fixes the MCS at activation, with no OLLA afterwards (<code>handle_ul_crc_info()</code> skips OLLA for <code>is_cg()</code>)
<code>dai</code> = 3, <code>tpc</code> = 1	Fields that exist only inside a DCI	Present on the activation DCI only. Periodic occasions carry no DCI.
<code>mimo</code> = 0, <code>ant</code> = 2	1 layer, DMRS antenna-port row 2	Captured once in the activation DCI (<code>antenna_ports</code>) and reused for every occasion

3.2 Expected log after implementation (illustrative)

These lines show the target format for the Phase 5 acceptance test. They are not captured output: the CS-RNTI value and slots are placeholders.

```
- UL PDCCH: rnti=<cs-rnti> type=cs-rnti ss_id=2 format=0_1 ... dci: act h_id=0 ndi=0 rv=0  
mcs=5 ...  
- UL PUSCH: rnti=<cs-rnti> h_id=<floor(sym/P) mod N> ... cg=Yes          (at n+k2, then every P  
slots)  
- MAC: rnti=<c-rnti> CG Confirmation MAC CE received  
- UL PDCCH: rnti=<cs-rnti> type=cs-rnti ... dci: rel h_id=0 ndi=0 rv=0 mcs=31 ...
```

4. What exists today (as-is)

4.1 Type 1 path, end to end

1. **Configuration.** The CLI `cell_cfg.cg` (`periodicity_slots`, `mcs`, `nof_harq_processes`, `max_nof_cg_rbs`, `grant_size/requested_bitrate`) is translated into `cg_builder_params`. `make_default_cg_config()` (`serving_cell_config_factory.cpp:261`) always builds a `cg_configuration` with `rrc_configured_ul_grant_cfg`. `make_default_cell_cg_config()` (`ran_cell_config_helper.cpp:444`) picks the TDRA row that avoids SRS symbols.
2. **Per-UE resources.** In the DU manager, `cg_type1_res_mng::alloc_resources()` (`cg_res_mng.cpp:124`) picks the least-loaded offset within the period and a contiguous VRB block. It stores them in the DU-internal `ue_cg_config {cg_offset, vrbs}` and writes them into the RRC grant.
3. **RRC and MAC.** The ASN.1 helper encodes `ConfiguredGrantConfig` and `rrc-ConfiguredUplinkGrant`. The MAC controller allocates the CS-RNTI when `cs_rnti_requested` is set and maps it to the UE for PUSCH decoding.
4. **Scheduler set-up.** `ue_event_manager` calls `cg_sched->add_reconf_ue()` (`ue_event_manager.cpp:397, :912`). `add_ue_to_wheel()` pushes the C-RNTI into `periodic_pusch_slot_wheel` at `time_domain_offset + k × P` and caches the TBS. `update_harq_reservation()` reserves N CG HARQs.
5. **Every slot.** `ue_scheduler_impl` runs the CG scheduler before the fallback and slice schedulers. `run_slot()` pre-reserves CG RBs in the UL grid (`reserve_*`). `allocate_cg_opportunity()` then computes the HARQ ID with `get_harq_id()`, allocates the HARQ with a CG timeout, and calls `build_pusch_cs_rnti()` (`is_cg = true`, `rnti = CS-RNTI`, `new_data = true`). UCI is multiplexed onto the CG PUSCH.
6. **Feedback.** `ue_cell::handle_ul_crc_info()` treats a CRC failure with SINR below `cg_pusch_sinr_threshold_dB` as DTX and skips OLLA for CG. CG has no retransmissions (`nof_harq_retx = 0`).

4.2 Places that assume Type 1

These are all the places at the baseline where a UE with `cg_cfg` present but `rrc_configured_ul_grant_cfg` absent is rejected, silently ignored or would crash.

#	Location (file:line)	What it assumes	Effect on a Type 2 UE today
1	<code>scheduler_ue_config_validator.cpp:29</code>	<code>VERIFY(... has_value(), "Only Configured Grant type 1 supported")</code>	UE configuration rejected
2	<code>serving_cell_config_factory.cpp:283-297</code>	<code>make_default_cg_config()</code> always emplaces the RRC grant	No way to express Type 2
3	<code>ran_cell_config_helper.cpp:455, :489</code>	Asserts in <code>make_default_cell_cg_config()</code> and <code>compute_nof_cg_prbs_per_ue()</code>	Abort
4	<code>cg_res_mng.cpp:164</code>	<code>ocudu_assert(... "must be set for a Type 1 CG")</code>	Abort
5	<code>configured_grant_scheduler_impl.cpp:72</code>	<code>update_harq_reservation()</code> reserves 0 HARQs without an RRC grant	No CG HARQ pool
6	<code>configured_grant_scheduler_impl.cpp:93</code>	<code>add_ue_to_wheel()</code> returns early	Never scheduled
7	<code>configured_grant_scheduler_impl.cpp:137</code>	<code>rem_ue()</code> returns early	Harmless today; must mirror the new add path
8	<code>configured_grant_scheduler_impl.cpp:34, :255, :284, :380</code>	<code>.value()</code> on the RRC grant in <code>compute_cg_vrbs</code> ,	Undefined behaviour or crash once the wheel is filled

#	Location (file:line)	What it assumes	Effect on a Type 2 UE today
		<code>reserve_cg_resources,</code> <code>build_cg_pusch_cfg_params,</code> <code>allocate_cg_opportunity</code>	
9	<code>sch_pdu_builder.cpp:680</code>	Assert in <code>build_pusch_cs_rnti()</code>	Crash on the first occasion
10	<code>ue_configuration.cpp:743</code>	<code>is_cg_slot()</code> : "TODO: support type 2"	Dynamic PUSCH not kept out of Type 2 occasions
11	<code>configured_grant_scheduler_impl.h:23</code>	Constructor takes no <code>pdccch_resource_allocator</code>	Cannot send an activation DCI
12	<code>dci_info.h:92</code> / <code>mac_cell_processor.cpp:472</code>	No CS-RNTI UL DCI type; <code>encode_dci()</code> calls <code>ocudu_terminate</code> on unknown types	Cannot build or encode the DCI
13	<code>lcid_ul_sch.h</code> / <code>pdu_rx_handler.cpp:288</code>	No LCID 55 entry; default branch logs "Unhandled LCID"	Confirmation ignored
14	<code>cg_configuration.h:170</code>	Remark "Type 2 CG is not currently supported"	Documentation only

4.3 Already type-agnostic, reused as-is

- `cg_configuration` struct: periodicity, `nof_harq_processes`, DMRS, MCS table, UCI beta offsets and the CG timer are shared by both types.
- ASN.1 ConfiguredGrantConfig encoder (`asn1_rrc_config_helpers.cpp:2951`): encodes `rrc-ConfiguredUplinkGrant` only if present.
- CS-RNTI life-cycle in MAC (`mac_controller::handle_ue_reconfiguration_request()`, `rnti_manager`) and the CS-RNTI → UE mapping used to decode CG PUSCH.
- `ue_cell::handle_reconfiguration_request()` reserves `nof_harq_processes` for any CG. `cell_harq_manager` handles CG allocation with `cg_harq_alloc_params` and the timeout.
- `get_harq_id()`, `periodic_pusch_slot_wheel`, `reserve_updated_ues_resources()`, `reserve_slot_cg_resources()`, `validate_cg_opportunity()`.
- `multiplex_uci_on_pusch()` (`configured_grant = true`), `ue_cell::handle_ul_crc_info()` (DTX detection, no OLLA for CG), and the PUSCH part of `scheduler_result_logger` (prints `cg=Yes`).
- `dci_0_1_configuration`, `dci_0_1_size` and `dci_0_1_pack()`: the wire format is the same for C-RNTI and CS-RNTI.

5. Parameters involved

5.1 ConfiguredGrantConfig (`cg\_configuration`)

Field	Values / current source	Type 1	Type 2	Change
periodicity	s1...s15120; CLI periodicity_slots → cg_builder_params.periodicity (default s140)	Used	Used identically	UNCHANGED
nof_harq_processes	1...16; CLI (default 4)	HARQ pool size	Same	UNCHANGED
mcs_table	qam64 (fixed in factory)	Applies	Applies (TS 38.214 §6.1.4.1)	UNCHANGED
cg_dmrs_cfg / trans_precoder	Type A DMRS; transform precoding off	Applies	Applies	UNCHANGED
uci_on_pusch_cfg	Semi-static beta offsets	Applies	Applies	UNCHANGED
configured_grant_timer	static constexpr 4	HARQ timeout 4 × P	Same	UNCHANGED
rep (repK, repK-RV)	static n1 / s1_0231	RV 0	RV 0	UNCHANGED
frequency_hopping, res_alloc	static disabled / type_1	-	Activation DCI: hopping flag 0, RA type 1	UNCHANGED
rrc_configured_ul_grant_cfg	optional	Present	Absent	MODIFY (who fills it, B2/C1)

5.2 Grant parameters: RRC field (Type 1) mapped to DCI field (Type 2)

RRC grant field (Type 1)	DCI 0_1 field (Type 2)	Proposed OCUDU source	Kept in
time_domain_offset	(none; implied)	Activation PDCCH slot n chosen so that $(n + k2) \bmod P = \text{ue_cg_config.cg_offset}$	type2_activation_ctx.wheel_offset
time_domain_allocation	time_resource	Template TDRA index from make_default_cell_cg_config(), looked up in the list the DCI indexes	tdra_idx
freq_domain_res	frequency_resource (RIV, N_BWP)	ue_cg_config.vrbs from cg_type1_res_mng	vrbs
antenna_port	antenna_ports	get_pusch_antenna_port_mapping_row_index(1 layer, ...), as the factory does today	antenna_port
dmrs_seq_initialization	dmrs_seq_initialization	0	-
precoding_and_nof_layers	precoding_info_nof_layers	0 (1 layer)	-
mcs	modulation_coding_scheme	cg_builder_params.mcs (CLI mcs, default 5)	mcs (+ TBS in ue_tbs_values)
srs_resource_indicator / frequency_hopping_offs	SRI / hopping flag	Unset / 0 (static in cg_configuration too)	-

RRC grant field (Type 1)	DCI 0_1 field (Type 2)	Proposed OCUDU source	Kept in
et			

5.3 DCI 0_1 fields: dynamic grant, activation, release, periodic occasion

Field	Bits	Dynamic C-RNTI (today)	Activation (CS-RNTI)	Release (CS-RNTI)	Periodic occasion
carrier_indicator / ul_sul / bwp_indicator	0-3 / 0-1 / 0-2	Unset	Unset	Unset	-
frequency_resource	RA type 1	RIV of scheduled VRBs	RIV of CG VRBs	all '1'	From cache
time_resource	0-4	TDRA row	CG TDRA row	Template (ignored)	From cache
frequency_hopping_flag	0-1	Unset	0	0	-
modulation_coding_scheme	5	Link adaptation	CG MCS	11111	Fixed (cache)
new_data_indicator	1	HARQ NDI	0 (validation)	0 (validation)	Always new tx
redundancy_version	2	HARQ	00 (validation)	00 (validation)	0
harq_process_number	4	Allocator	0000 (validation)	0000 (validation)	floor(sym/P) mod N
first / second DAI	1-2 / 0-2	Per slot / unset	As for a dynamic grant (verify UE use)	0	-
tpc_command	2	Power control	Power control (applied once)	1 (0 dB)	-
antenna_ports / precoding / dmrs_seq_init	2-5 / var / 0-1	DMRS / MIMO	Template values	Template (ignored)	From cache
srs_request / csi_request	2-3 / 0-6	Per slot / unset	0 / 0	0 / 0	-
SRI / cbg / ptrs_dmrs / beta_offset	var	Unset	Unset	Unset	-

5.4 New runtime state: `type2\_activation\_ctx` (one per UE)

This is the "activation cache" from the first draft, made concrete. It lives in `configured_grant_scheduler_impl` (for example a `slotted_id_table<du_ue_index_t, type2_activation_ctx, MAX_NOF_DU_UE>`, like `ue_tbs_values`). It is runtime state, not RRC configuration.

Field	Type	Meaning
state	enum {idle, activating, active, releasing}	Life-cycle state machine (Figure 4)
activation_slot / wheel_offset	slot_point / unsigned	PDCCH slot n, and (n + k2) mod P used by <code>add_ue_to_wheel()</code> and <code>rem_ue()</code>
tdra_idx, symbols, k2	uint8_t / ofdm_symbol_range / unsigned	Row of the TDRA list the DCI indexed

Field	Type	Meaning
vrbs	vrbs_interval	CG frequency allocation
mcs, antenna_port	sch_mcs_index / uint8_t	Fixed after activation
nof_dci_tx, last_dci_slot	unsigned / slot_point	Retransmission of the activation or release DCI until the CG Confirmation arrives

5.5 New configuration parameters

Parameter	Where	Values / default	Purpose
cell_cfg.cg.type	CLI → du_high_configured_grants::type → cg_builder_params::cg_type	{1, 2}, default 1	Selects Type 1 or Type 2; the default keeps current behaviour
cg_type2_activation_retx_slots (proposed)	scheduler_ue_expert_config	e.g. 2 × P (min 20 slots)	Wait before re-sending an unconfirmed activation or release DCI
cg_type2_max_dci_attempts (proposed)	scheduler_ue_expert_config	e.g. 4	After this many attempts: give up, log a warning, fall back to dynamic grants

6. Proposed changes: file and function inventory

The IDs match the boxes in Figures 1–3, so each row can be found in the diagrams. Paths are relative to the OCUDU root; `lib/scheduler/configured_grant/` is shortened to `cg/`.

6.1 Configuration path (Figure 1)

ID	File	Function / symbol	Action	Change and reason
A. App / CLI (apps/units/flexible_o_du/o_du_high/du_high/)				
A1	<code>du_high_config.h:573</code>	<code>du_high_configured_grants</code>	MODIFY	Add unsigned type = 1.
A2	<code>du_high_config_cli11_schema.cpp:1452</code>	<code>configure_cli11_cg_args()</code>	MODIFY	Add <code>--type</code> with <code>check(CLI::IsMember({1, 2}))</code> .
A3	<code>du_high_config_translators.cpp:1071</code>	CG block building <code>du_cg_params</code>	MODIFY	Copy type into <code>cg_builder_params::cg_type</code> .
A4	<code>du_high_config_validator.cpp</code>	CG validation	MODIFY	Check the value; for Type 2 require a UE-specific search space that monitors DCI 0_1.
B. Cell & UE configuration builders				
B1	<code>include/ocudu/scheduler/config/cg_builder_params.h:14</code>	<code>cg_builder_params</code>	MODIFY	Add enum class <code>cg_type_t { type1, type2 } cg_type = type1;</code> .
B2	<code>lib/scheduler/config/serving_cell_config_factory.cpp:261</code>	<code>make_default_cg_config()</code>	MODIFY	Emplace <code>rrc_configured_ul_grant_cfg</code> only for Type 1. For Type 2, keep the grant defaults (MCS, antenna port) in the template.
B3	<code>lib/scheduler/config/ran_cell_config_helper.cpp:444, :486</code>	<code>make_default_cell_cg_config()</code> , <code>compute_nof_cg_prbs_per_ue()</code>	MODIFY	Replace the Type-1 asserts. Compute TDRA and PRB count from the template MCS/TDRA so both types share RB planning.
B4	<code>include/ocudu/ran/configured_grant/cg_configuration.h</code>	<code>cg_configuration</code>	UNCHANGED	Only update the remark at line 170 ("Type 2 CG is not currently supported").
B5	<code>lib/scheduler/config/scheduler_ue_config_validator.cpp:29</code>	CG UE validation	REMOVE	Delete <code>VERIFY(... "Only Configured Grant type 1 supported")</code> and make the TDRA check type-aware. Add a Type 2 check that a CS-RNTI and a DCI 0_1 USS exist.
C. DU manager / RRC / MAC control				
C1	<code>lib/scheduler/rrm/cg_res_mng.cpp:124, :189</code>	<code>cg_type1_res_mng::alloc_resources()</code> / <code>dealloc_resources()</code>	MODIFY	Keep offset/VRB planning for both types. Write them into the RRC grant only for Type 1. Remove the assert at :164. Optionally rename the class to <code>cg_res_mng</code> .
C2	<code>include/ocudu/scheduler/config/ue_bwp_config.h:52</code>	<code>ue_cg_config</code>	MODIFY	Add <code>mcs</code> and <code>tdra_idx</code> (with <code>operator==</code>). This becomes the DU-internal Type 2 grant template the scheduler activates.
C3	<code>lib/du/du_high/du_manager/</code>	<code>ConfiguredGrantConfig</code>	UNCHANGED	Already omits <code>rrc-ConfiguredUplinkGrant</code> when

ID	File	Function / symbol	Action	Change and reason
	converters/ asn1_rrc_config_helpers.cpp:2951	encoder		absent.
C4	lib/mac/mac_ctrl/ mac_controller.cpp:109	handle_ue_reconfiguration_request()	UNCHANGED	CS-RNTI allocation depends only on cs_rnti_requested.
C5	OAI nr-uesoftmodem	UE MAC (CG handling)	VERIFY	Must accept a CG without rrc-ConfiguredUplinkGrant, validate the CS-RNTI DCI and send LCID 55. Unconfirmed (Risk R1).
D. Scheduler: UE add / reconfiguration				
D1	lib/scheduler/ ue_scheduling/ ue_event_manager.cpp:397, :912	calls to cg_sched->add_reconf_ue()	UNCHANGED	Entry points stay the same.
D2	cg/ configured_grant_scheduler_impl.cpp:165	add_reconf_ue()	MODIFY	For Type 2: create <code>type2_activation_ctx{state = idle}</code> and put the UE on the pending-activation list instead of filling the wheel. On reconfiguration of an active Type 2 UE, schedule a release first.
D3	cg/ configured_grant_scheduler_impl.cpp:59	update_harq_reservation()	REMOVE	Drop the <code>rrc_configured_ul_grant_cfg.has_value()</code> term so Type 2 UEs keep their N reserved CG HARQs from configuration time (decision D6).
D4	lib/scheduler/ ue_context/ ue_cell.cpp:60	ue_cell::handle_reconfiguration_request()	UNCHANGED	Already reserves CG HARQs for any CG type.
D5	cg/ configured_grant_scheduler_impl.h	pending-activation list	OUTPUT	Hand-off to Figure 2.

6.2 Activation and release path (Figure 2)

ID	File	Function / symbol	Action	Change and reason
E. Slot entry & activation control				
E1	lib/scheduler/ ue_scheduling/ ue_scheduler_impl.cpp:213	cell constructor (cg_sched)	MODIFY	Pass <code>*params.pdcch_sched</code> (and the expert config) to <code>configured_grant_scheduler_impl</code> . The CG scheduler still runs before the fallback and slice schedulers.
E2	cg/ configured_grant_scheduler_impl.cpp:188	run_slot()	MODIFY	Call <code>schedule_type2_activations(cell_alloc)</code> before <code>allocate_slot_cg_opportunities()</code> .
E3	cg/ configured_grant_scheduler_impl.cpp:h	schedule_type2_activations()	NEW	For each UE in idle/activating/releasing: if $(\text{pdcch_slot} + k2) \bmod P == \text{cg_offset}$, UL is enabled at the PUSCH slot and the retry timer has

ID	File	Function / symbol	Action	Change and reason
				expired, then allocate the PDCCH (F1), build the DCI (F2), set the state, and for activation call <code>add_ue_to_wheel()</code> so the first occasion at $n + k_2$ is reserved and decoded.
E4	cg/ configured_grant_scheduler_impl.h	type2_activation_ctx	NEW	Runtime cache and state machine (Section 5.4).
E5	cg/ configured_grant_scheduler_impl.cpp	schedule_type2_release()	NEW	Triggered by <code>rem_ue()</code> , CG reconfiguration or (optionally) inactivity. Sends the release DCI and removes the UE from the wheel. On UE deletion the release is best-effort.
E6	cg/ configured_grant_scheduler_impl.cpp	get_active_grant()	NEW	Returns the resolved grant (TDRA, symbols, VRBs, MCS). Type 1 reads the RRC grant; Type 2 reads the cache, or nothing if not active. Used by H4, H5, I1 and I2.
F. DCI build / PDCCH / MAC-DL				
F1	lib/scheduler/ pdcch_scheduling/ pdcch_resource_allocator_impl.cpp:98-138	alloc_ul_pdcch_ue()	MODIFY	Allocate with the C-RNTI (USS candidates and <code>n_RNTI</code> data scrambling, TS 38.211 §7.3.2.3), then set <code>ctx.rnti</code> (the CRC RNTI) to CS-RNTI. Add an optional <code>crc_rnti</code> parameter rather than overwriting ad hoc.
F2	lib/scheduler/support/ dci_builder.cpp/h	build_dci_f0_1_cs_rnti()	NEW	Signature like <code>build_dci_f0_1_c_rnti()</code> but with no <code>ul_harq_process_handle</code> , plus <code>bool release</code> . Activation: NDI 0, HARQ 0, RV 0, grant fields from the template. Release: also MCS 31 and FDRA all ones. Payload size = <code>format0_1_ue_size</code> .
F3	lib/scheduler/support/ dci_builder.cpp:383	build_dci_f0_1_c_rnti()	UNCHANGED	Reference for field sourcing (antenna ports, precoding, TPC, DAI).
F4	include/ocudu/ scheduler/result/ dci_info.h:92-129	dci_ul_rnti_config_type, dci_ul_info	MODIFY	Add <code>cs_rnti_f0_1</code> to the enum and a matching variant alternative (a wrapper type). Add <code>as_/set_cs_rnti_f0_1()</code> , extend <code>rnti_types</code> to "cs-rnti", and make <code>get_dci_format()</code> return <code>f0_1</code> for it.
F5	lib/mac/mac_dl/ mac_cell_processor.cpp:472	encode_dci() (UL)	MODIFY	Add case <code>cs_rnti_f0_1</code> : return <code>dci_0_1_pack(...)</code> . <code>dci_0_1_pack()</code> itself is UNCHANGED.
F6	lib/scheduler/logging/ scheduler_result_logger.cpp:173	UL PDCCH logging switch	MODIFY	Print <code>type=cs-rnti</code> and act/rel so the lab test can confirm the DCI.
G. UE feedback: CG Confirmation MAC CE				

ID	File	Function / symbol	Action	Change and reason
G1	OAI nr-uesoftmodem	UE	VERIFY	See C5.
G2	lib/mac/mac_ul/ lcid_ul_sch.h	lcid_ul_sch_t::options	MODIFY	Add CG_CONFIRMATION = 0b110111. sizeof_ce() returns 0 (correct). No length field.
G3	lib/mac/mac_ul/ pdu_rx_handler.cpp:216-288	pdu_rx_handler::handle_mac_ce()	MODIFY	New case: forward {ue_index, cell, slot_rx} to the scheduler instead of "Unhandled LCID".
G4	include/ocudu/ scheduler/ scheduler_feedback_handler.h:276 (+ ue_event_manager, ocudu_scheduler_adapter)	handle_cg_confirmation_indication()	NEW	New feedback API, queued and routed per cell like handle_ul_ta_report_indication().
G5	cg/ configured_grant_scheduler.h / _impl.cpp	handle_cg_confirmation()	NEW	activating → active, releasing → idle (context erased). Stops DCI retransmission.

6.3 Periodic-occasion path (Figure 3)

ID	File	Function / symbol	Action	Change and reason
H. CG scheduler: slot wheel & reservation (cg/configured_grant_scheduler_impl.cpp)				
H1	:79	add_ue_to_wheel()	MODIFY	Replace the early return with a type branch. Type 1: offset = time_domain_offset (unchanged). Type 2: called from E3 with offset = (n + k2) mod P and the TBS from the cache.
H2	configured_grant_scheduler_impl.h	periodic_pusch_slot_wheel	UNCHANGED	5120-slot ring, which fits every periodicity.
H3	:204, :228	reserve_updated_ues_resources(), reserve_slot_cg_resources()	UNCHANGED	Only iterate the wheel.
H4	:240	reserve_cg_resources()	MODIFY	Read TDRA symbols and VRBs from get_active_grant().
H5	:124, :268	rem_ue(), stop()	MODIFY	rem_ue() uses the active offset (the Type 2 cache), skips never-activated UEs and triggers E5. stop() also clears the Type 2 contexts.
I. Per-occasion PUSCH build & HARQ				
I1	cg/...impl.cpp:365	allocate_cg_opportunity()	MODIFY	MCS and VRBs from get_active_grant(); skip if the grant is not active. validate_cg_opportunity() (:331) is UNCHANGED.
I2	cg/...impl.cpp:277, :31	build_cg_pusch_cfg_params(), compute_cg_vrbs()	MODIFY	Take the resolved grant as input. Use the TDRA list the activation DCI indexed (see Risk R2).

ID	File	Function / symbol	Action	Change and reason
I3	cg/...impl.cpp:19; lib/scheduler/cell/cell_h arq_manager.cpp	get_harq_id(), alloc_ul_harq()	UNCHANGED	Same HARQ formula and CG timeout for Type 2.
I4	lib/scheduler/support/ sch_pdu_builder.cpp:68 0	build_pusch_cs_rnti()	REMOVE	Delete the Type-1-only assert, or change it to check that the CG configuration exists. The body only uses resolved values passed in, so it needs no other change.
I5	include/ocudu/ scheduler/result/ pusch_info.h	pusch_information (is_cg = true)	OUTPUT	CS-RNTI PUSCH every P slots until release.

J. Interaction with the dynamic scheduler

J1	lib/scheduler/config/ ue_configuration.cpp:7 43	ue_cell_configuration: :is_cg_slot()	MODIFY	Keep it as the Type 1 (config-only) check. Add a runtime query for Type 2, for example <code>ue_cell::is_cg_slot(slot)</code> that uses a flag and offset set by the CG scheduler on activation or release.
J2	grant_params_selector. cpp:419; ue_fallback_scheduler. cpp:535, 554, 1059	is_cg_slot() call sites	MODIFY	Switch to the runtime query so dynamic PUSCH and PUCCH/UCI placement avoid active Type 2 occasions only.
J3	lib/scheduler/ uci_scheduling/ uci_allocator_impl.cpp	multiplex_uci_on_pusch ()	UNCHANGED	Already called with <code>configured_grant = true</code> .
J4	lib/scheduler/ ue_context/ ue_cell.cpp:155	ue_cell::handle_ul_crc _info()	UNCHANGED	DTX detection and fixed MCS keyed on <code>is_cg()</code> .
J5	lib/scheduler/logging/ scheduler_result_logge r.cpp:675	PUSCH logging	UNCHANGED	Already prints cg=Yes.

6.4 Tests

ID	File	Test	Action	What it checks
T1	tests/unittests/scheduler/ cg_scheduler/ configured_grant_scheduler_ test.cpp	new <code>cg_type2_*</code> cases in the <code>cg_duplex_test</code> suite	MODIFY	No PUSCH before activation. Activation DCI (CS-RNTI, NDI 0, HARQ 0, RV 0) at slot n with $(n + k_2) \bmod P = \text{offset}$. Periodic PUSCH from $n + k_2$. Re-send without confirmation. Release DCI and no PUSCH afterwards. FDD and TDD.
T2	tests/unittests/scheduler/... (dci_builder test)	<code>build_dci_f0_1_cs_rnti</code> activation/release	NEW	Bit patterns from TS 38.213 Tables 10.2-3/4. Packed size equals the C-RNTI DCI 0_1.
T3	tests/unittests/mac/... (mac_ul_pdu test)	LCID 55 parsing	NEW	A PDU with CG Confirmation plus data parses, and the CE reaches the scheduler.
T4	existing CG, validator, asn1 tests	regression	UNCHANGED	Must still pass after Phase 0 (Type 1 behaviour)

ID	File	Test	Action	What it checks
				unchanged).

6.5 Summary counts

Action	Count	IDs
NEW	12	E3, E4, E5, E6, F2, G4 (API + event-manager route + adapter), G5 (interface + impl), T2, T3
MODIFY	27	A1-A4, B1-B3, C1, C2, D2, E1, E2, F1, F4, F5, F6, G2, G3, H1, H4, H5, I1, I2, J1, J2, T1 (+ B4 remark)
REMOVE	3	B5 (validator VERIFY), D3 (HARQ guard), I4 (PUSCH builder assert)
UNCHANGED (reused)	16	B4, C3, C4, D1, D4, F3, dci_0_1_pack/size, H2, H3, I3, J3, J4, J5, validate_cg_opportunity, CS-RNTI table, T4
VERIFY (external)	1	C5 / G1: OAI UE

7. Block diagrams

Figures 1–3 show every file and function the proposal touches, grouped by execution path and colour-coded (green = new, amber = modify, red = remove or relax a Type-1 guard, grey = reused unchanged, dashed purple = verify / external, blue = output or hand-off). Box IDs match the tables in Section 6. Figure 4 shows the signalling sequence and which function handles each message. The four figures follow on landscape pages.

Figure 1 - Configuration path: files and functions touched for CG Type 2

UE setup / RRC reconfiguration: from the CLI option to the scheduler's pending-activation queue

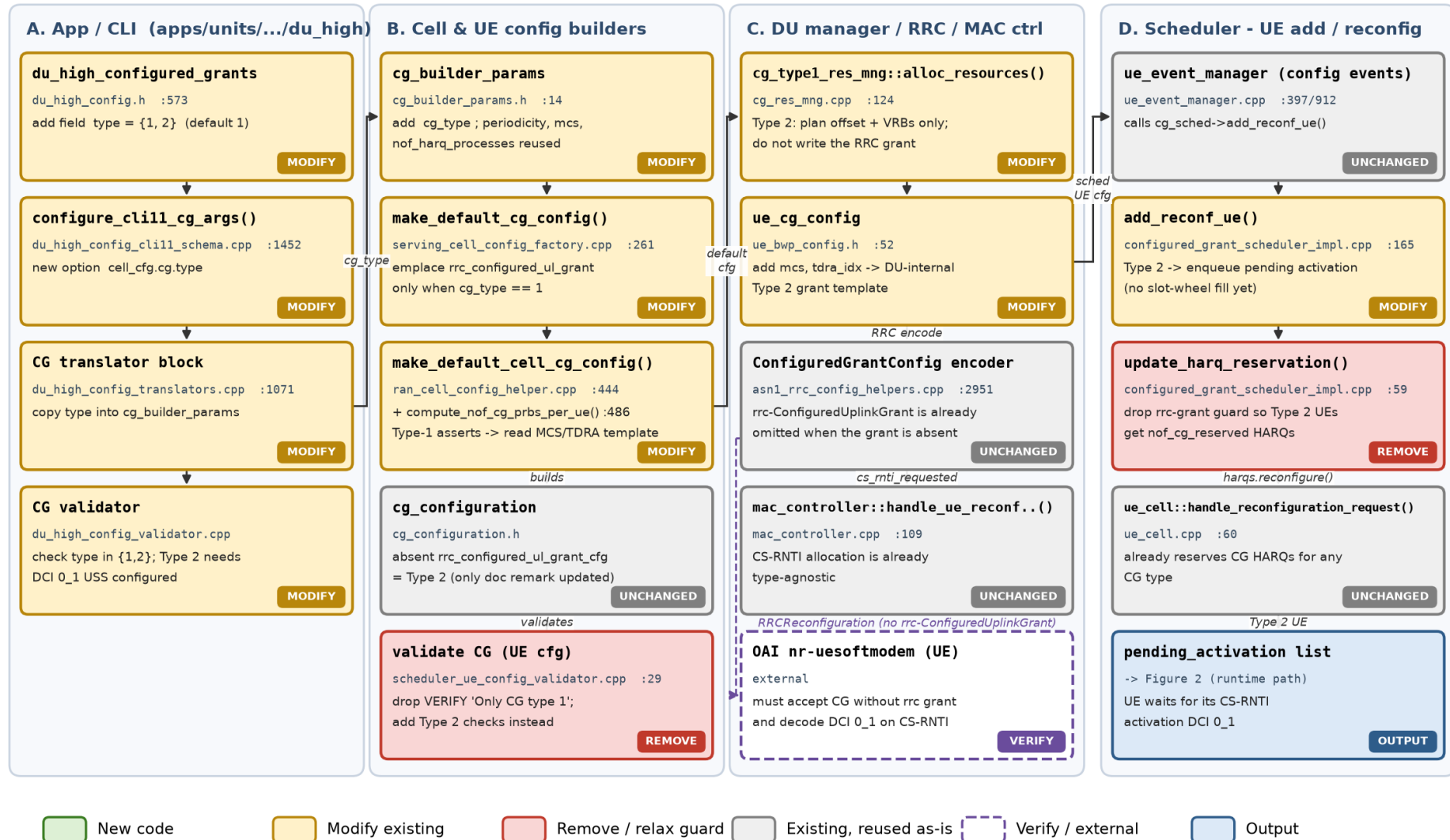


Figure 1: configuration path (IDs A1-D5 in Section 6.1).

Figure 2 - Runtime path A: Type 2 activation / release (DCI 0_1 on CS-RNTI)

What runs once per activation or release. Everything in this figure is Type-2-only; Type 1 never enters it.

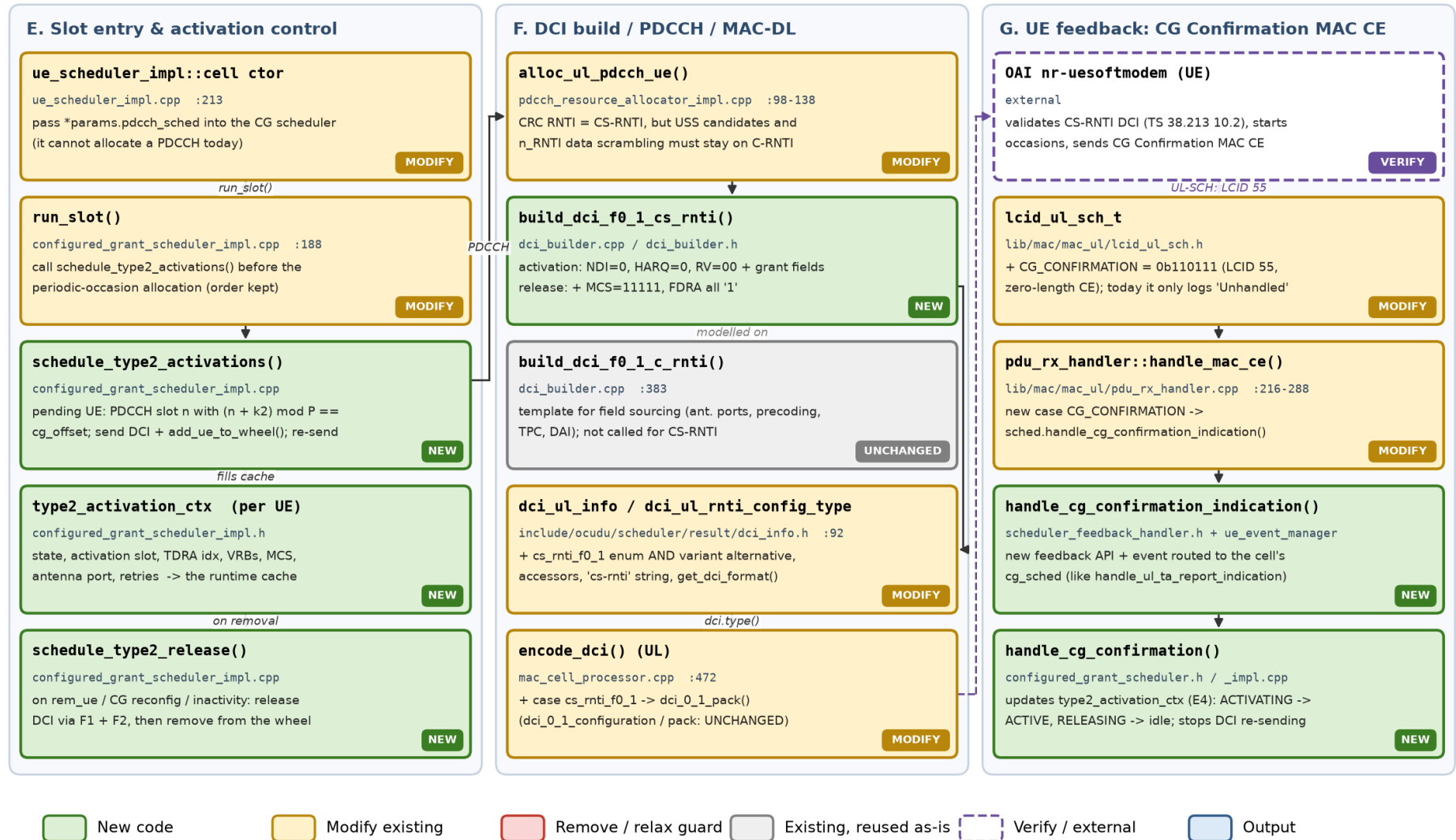


Figure 2: activation and release path (IDs E1-G5 in Section 6.2). E6 get_active_grant() is used by Figure 3.

Figure 3 - Runtime path B: periodic CG occasions (shared by Type 1 and Type 2)

What runs every slot. The Type 1 path already works; the changes let the same code read a DCI-activated grant.

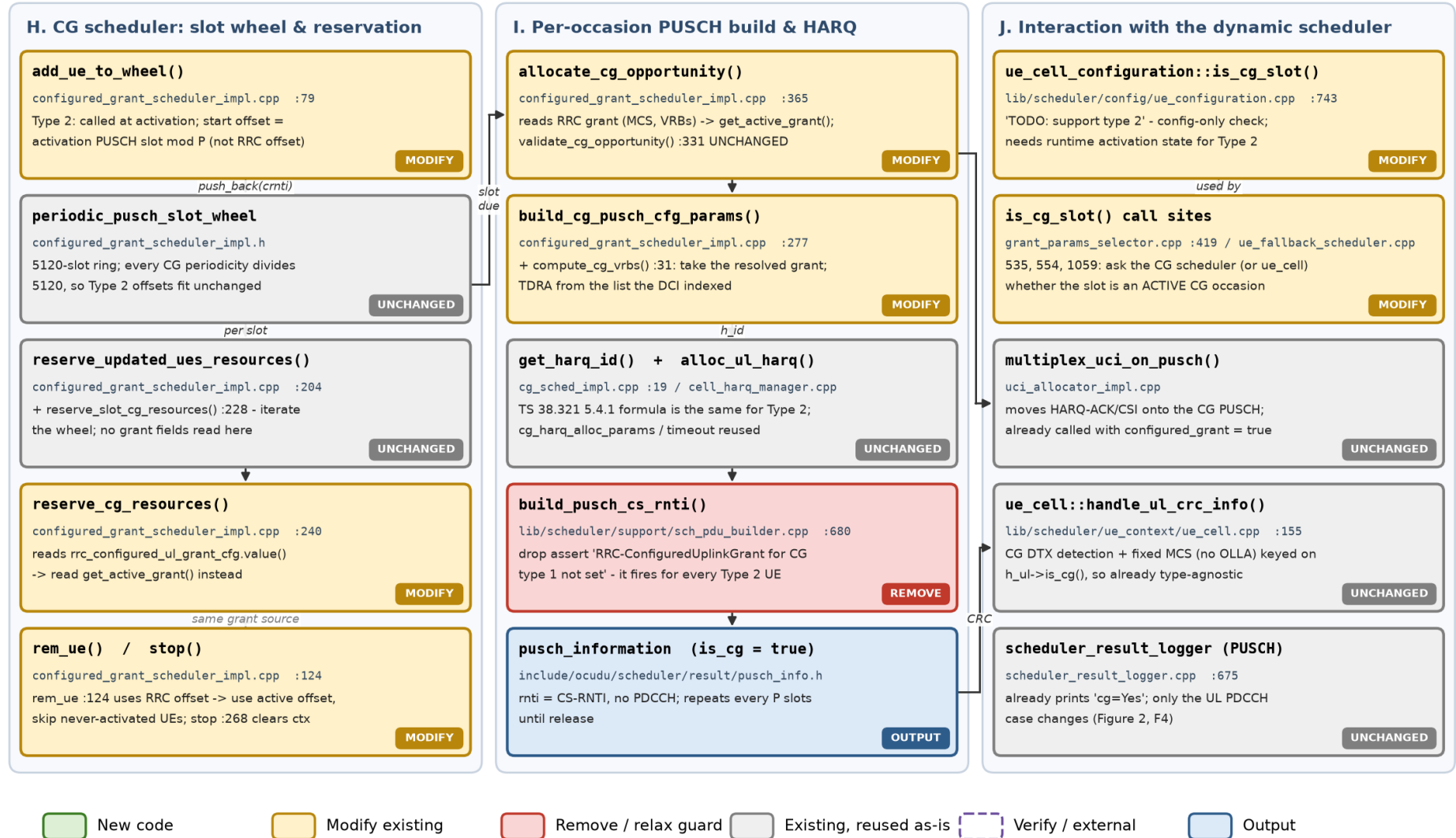


Figure 3: periodic-occasion path shared by both types (IDs H1-J5 in Section 6.3).

Figure 4 - CG Type 2 lifecycle: signalling sequence and the OCUDU function that handles each step

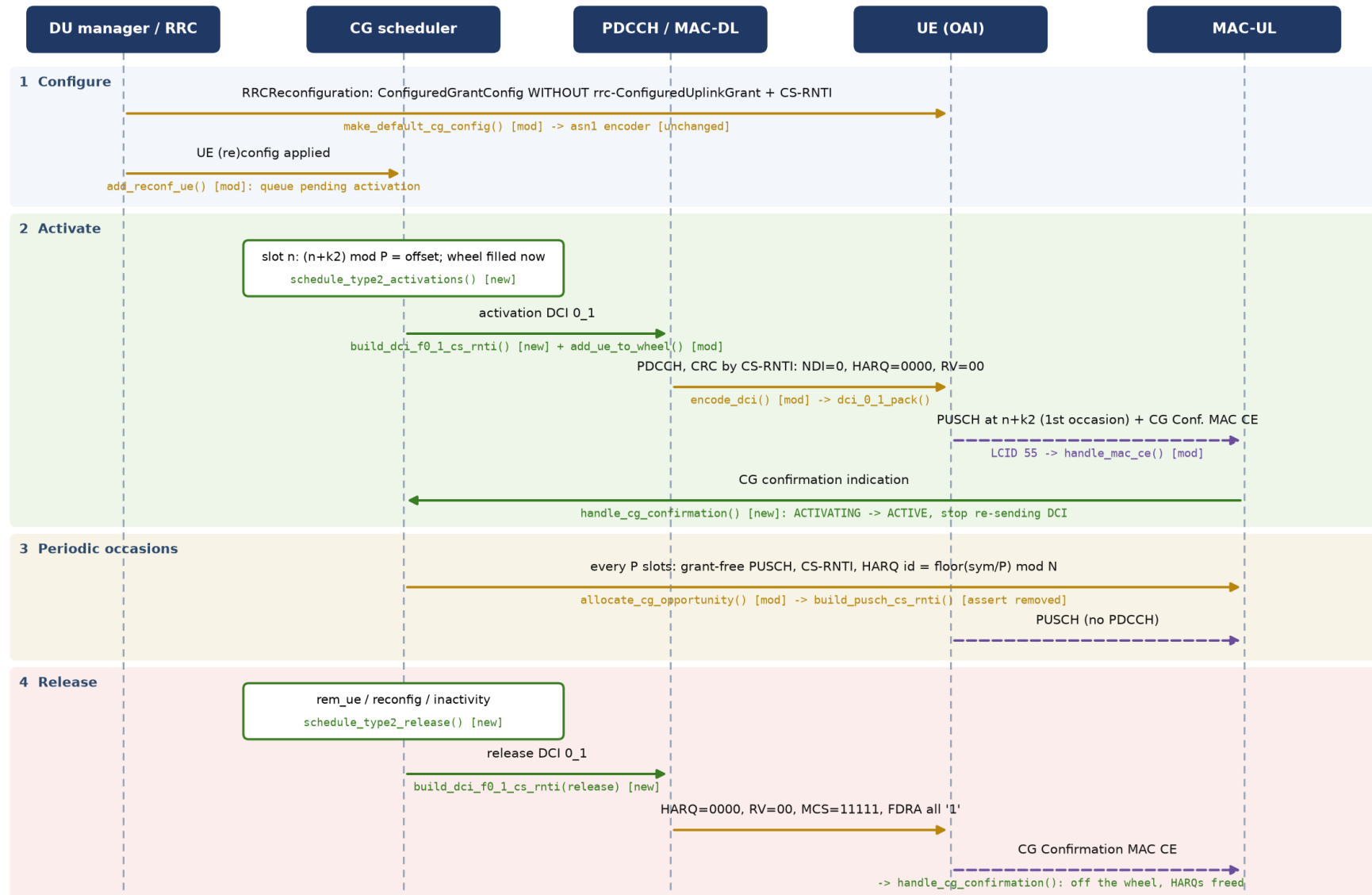


Figure 4: Type 2 lifecycle. [new] / [mod] tags match the colour code.

8. Design decisions

#	Decision	Recommendation	Why / alternative rejected
D1	Where the Type 2 grant comes from	Reuse <code>cg_type1_res_mng</code> planning (offset + VRBs) and store it as a DU-internal template (<code>ue_cg_config</code> + MCS/TDRA).	Keeps the existing RB-collision planning with PRACH/PUCCH valid. Picking resources freshly at activation time would need a second allocator.
D2	Which slot to send the activation DCI in	Wait until $(n + k_2) \bmod P = \text{planned offset}$.	The occasions then land exactly on pre-planned, non-colliding RBs. Worst-case delay is P slots, once per activation.
D3	Representing the CS-RNTI DCI	New variant alternative with a distinct wrapper type.	<code>type()</code> comes from the variant index. Reusing <code>dci_0_1_configuration</code> twice breaks <code>std::get<T></code> . Keeping type <code>c_rnti_f0_1</code> and changing only <code>ctx.rnti</code> would mislabel logs and hide activation DCIs from tests.
D4	PDCCH RNTI handling	Search-space candidates and <code>n_RNTI</code> from C-RNTI; CRC RNTI = CS-RNTI (new optional parameter to <code>alloc_ul_pdcch_ue()</code>).	TS 38.211 §7.3.2.3 and TS 38.213 §10.1 tie USS hashing and scrambling to the C-RNTI.
D5	When to fill the slot wheel	When the activation DCI is sent, not when the confirmation arrives.	The UE starts transmitting at $n + k_2$, usually carrying the confirmation in that first PUSCH. If the confirmation doesn't arrive in time, re-send the activation.
D6	When to reserve CG HARQs	At configuration time for both types (drop the D3 guard).	Avoids reconfiguring the HARQ entity at runtime. The cost is N idle HARQ IDs before activation.
D7	<code>is_cg_slot</code> for Type 2	Runtime flag and offset on <code>ue_cell</code> , set by the CG scheduler.	<code>ue_cell_configuration</code> is immutable config and cannot know the activation state.
D8	Release triggers	UE removal, CG reconfiguration, and optional inactivity (no BSR for X ms).	Inactivity-based release is what makes Type 2 useful (saves resources); keep it behind a config flag.

9. Implementation plan and test plan

Phase	Scope (IDs)	Deliverable	Exit criterion
0. Refactor	E6, H4, H5, I1, I2 (Type 1 only)	<code>get_active_grant()</code> ; no <code>.value()</code> on the RRC grant outside it	All existing CG tests pass; no behaviour change (can merge alone)
1. Configuration	A1-A4, B1-B5, C1, C2, D2 (queue only), D3	Type 2 UE config accepted; RRC without <code>rrc-ConfiguredUplinkGrant</code>	Validator and ASN.1 unit tests; the UE attaches with Type 2 configured (no activation yet)
2. DCI	F1, F2, F4, F5, F6	CS-RNTI DCI 0_1 can be built, encoded and logged	T2 passes; packed size equals C-RNTI DCI 0_1
3. Scheduler	E1-E5, H1, J1, J2	Activation and release state machine; Type 2 occasions scheduled	T1 passes (FDD and TDD)
4. Confirmation	G2-G5	LCID 55 routed to the CG scheduler	T3 passes; no "Unhandled LCID" warning

Phase	Scope (IDs)	Deliverable	Exit criterion
5. End-to-end	C5/G1 check, lab	ZMQ run with OAI UE	Log lines as in Section 3.2; iperf UL over CG occasions; clean release

10. Risks and open questions

#	Risk / question	Impact	Mitigation
R1	OAI UE support for Type 2 (CS-RNTI DCI validation, LCID 55) is unverified.	High: blocks Phase 5	Read the OAI UE MAC CG code before Phase 3. If missing, test with srsUE/COTS or patch OAI.
R2	TDRA list mismatch: Type 1 indexes the common <code>pusch_td_alloc_list</code> , but DCI 0_1 in a USS indexes the dedicated list if one is configured.	Medium: wrong symbols or k2	Resolve TDRA through <code>ss_info.bwp->ul.td_mapper()</code> for Type 2 and assert the template index exists there.
R3	PDCCH blocking delays activation (the aligned slot occurs only once per P).	Low	Retry at the next aligned slot; count attempts (<code>cg_type2_max_dci_attempts</code>).
R4	CG retransmission via CS-RNTI with NDI = 1 is not supported for either type (<code>nof_harq_retx = 0</code>).	Low (existing limitation)	Out of scope; note as follow-up.
R5	DAI and UCI semantics on the activation DCI's first PUSCH.	Low	Compute DAI as for a dynamic grant; confirm UE behaviour in the lab.
R6	Does <code>pdcch_ul_information::ctx.rnti</code> feed anything other than CRC scrambling (for example the FAPI PDU RNTI or UE lookups)?	Medium	Audit the MAC→FAPI translator in Phase 2 before changing <code>ctx.rnti</code> .
R7	Multiple CG configurations (Rel-16) and <code>harq-ProclD-Offset2</code>	None now	Out of scope; the HARQ field validation assumes a single configuration.
R8	DRX or measurement gaps at the aligned activation slot	Low	<code>is_ul_enabled()</code> and active-time checks before sending; otherwise wait for the next aligned slot.

11. Glossary

Term	Meaning
CG Type 1 / Type 2	Configured Grant. Type 1: the whole grant comes from RRC. Type 2: RRC gives the semi-static part, and a CS-RNTI DCI activates or releases the grant and carries its fields.
C-RNTI / CS-RNTI	Cell RNTI (dynamic scheduling) / Configured Scheduling RNTI (CG activation, release, retransmission and CG PUSCH)
Activation / release DCI	DCI 0_1 with CRC scrambled by CS-RNTI, NDI = 0, and the reserved values of TS 38.213 §10.2
CG Confirmation MAC CE	Zero-length UL MAC CE (LCID 55) that the UE sends after activation or release
NDI, RV, HARQ ID	New Data Indicator, Redundancy Version, HARQ process number. All three are validation fields in activation and release DCIs.
TDRA / RIV / VRB	Time-domain resource allocation row / Resource Indication Value (RA type 1) / Virtual Resource Block
P, N, k2	CG periodicity in slots, <code>nrofHARQ-Processes</code> , PDCCH-to-PUSCH slot offset
<code>periodic_pusch_slot_wheel</code>	The CG scheduler's 5120-slot ring of UEs with a CG occasion in each slot

Term	Meaning
type2_activation_ctx	Proposed per-UE runtime cache and state machine for Type 2 (Section 5.4)
USS	UE-specific search space (DCI 0_1 is monitored only there)